

CS6600: Computer Architecture

Assignment #3

Dynamic Scheduling

Name: **Dinesh Pitchika**

Roll No: **EE24M102**

1 Introduction

This project involved creating a mock simulator of an Out of Order Superscalar Processor in C++ using Tomasulo's Algorithm for dynamic scheduling. The purpose of this project is to just look at how IPC(Instructions per Cycle) is enhanced by scaling a simple Out-of-Order CPU interms of fetching N instructions at a time and also Scheduling S instructions. Precise interrupts are not guaranteed in this implementation.

2 Experiments

In this section, the trace provided in the *gcc_trace.txt* and *perl_trace.txt* files was run with different values of Scheduling Queue Size(S) and Fetch Size(N). Variations in IPC for different values of 'N' and 'S' are being examined.

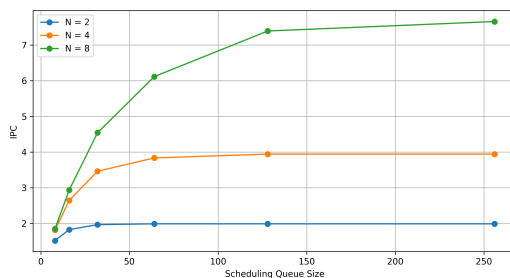


Figure 1: IPC vs S for different N - GCC Trace

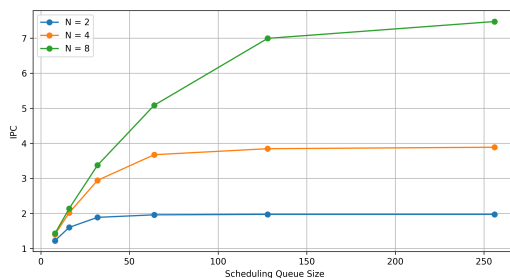


Figure 2: IPC vs S for different N - PERL Trace

From the plots for both traces we can observe the following,

- For a given **N**, IPC increases initially for smaller values of **S**, but after a certain point **IPC** gets saturated close to **N** which is ideally the maximum IPC that can be achieved.
- It indicates that for smaller values of **S** because of limited capacity to track and reorder dependent instructions the CPU keeps frequently stalling whenever there are dependent instructions which leads to low IPC.
- As **S** increases, the CPU can now track more instructions and schedule independent instructions which increases the IPC.
- As **S** increases beyond a certain point IPC starts to saturate because we are only fetching **N** instruction despite having a larger Queue for scheduling and IPC can't go beyond **N**.
- For a given **S**, as **N** increases we can observe that IPC increases due to the fact that more instructions are being fetched and the CPU gets the opportunity to schedule more independent instructions.
- For a given value of **N**, we know that maximum IPC that can be achieved is **N**.
- However if we compare the saturated value of IPC for each value of **N**, we can observe that the value keeps going further away from **N**. For Example, for **N=2**, saturated IPC is **1.98** and for **N=8** saturated IPC is **7.66**. It can be because of the fact that no matter how much hardware we provide for parallelisation, the program or the trace have dependent instructions which limits the extent to which those instructions can be executed in parallel.

Comparing the IPC values for both **GCC** and **PERL** traces, we can observe that the trends are similar and the above arguments hold for both of them. However if we observe closely, for given values of **N** and **S**, **IPC for GCC is more than for the PERL trace**. We can see this particularly for the saturated values of IPC for each **N**. This might indicate that **PERL trace has more number of dependent instructions than the GCC trace** and therefore low IPC.